

NUEVAMENTE

Plan de Acción

Equipo de Ciencia de Datos

Hackathon ONE G10 - Sistema Inteligente de Adaptación y Generación de Contenido Educativo

Horizonte	Objetivo técnico	Resultado esperado
5 semanas	Base documental confiable + pipeline RAG evaluable	MVP integrado, reproducible y demostrable

Misión del área

Diseñar y validar la capa de conocimiento del proyecto: fuentes, datos, metadatos, procesamiento, recuperación RAG, adaptación pedagógica y evaluación de fidelidad. El foco inicial debe ser un sistema confiable antes que un gran volumen de documentos.

Temas iniciales: IA · Backend · Data Science · Frontend · Cloud/DevOps

Público objetivo: desde estudiantes de escuela hasta profesionales del área tecnológica, con adaptación del nivel y formato pedagógico.

1. Qué debe construir el equipo de Ciencia de Datos

El reto plantea una pipeline de RAG con ingestión de documentos, chunking, embeddings, búsqueda vectorial, orquestación con LLM y mecanismos para verificar fidelidad al documento de origen. Por ello, el trabajo debe organizarse como un sistema de conocimiento y evaluación, no como un problema clásico de clasificación.

Pipeline objetivo

Fuentes → documentos → limpieza → metadatos → chunks → embeddings → Vector Store → retrieval → LLM → adaptación por audiencia → evaluación

Regla principal

Primero construir una colección pequeña y confiable, con criterios de calidad y evaluación. Después ampliar el corpus. La calidad del retrieval importa más que la cantidad bruta de archivos.

Decisiones que deben quedar cerradas en la primera semana

- Taxonomía de temas, subtemas y tecnologías.
- Niveles de audiencia y criterios de adaptación.
- Esquema de metadatos y catálogo de documentos.
- Criterios para aceptar o rechazar una fuente.
- Estrategia inicial de chunking y embeddings.
- Vector Store y método de retrieval.
- Dataset de evaluación y métricas.
- Contrato de salida JSON para integración con backend.

2. Taxonomía recomendada

La categoría debe describir el área principal del documento, mientras que subcategoría, tema y tecnología reducen ambigüedades. No conviene usar una sola etiqueta, porque muchos contenidos se solapan entre IA, Data Science y Backend.

Categoría	Subcategorías sugeridas	Ejemplos
IA	IA generativa, LLM, agentes, RAG, NLP, visión	Transformers, embeddings, prompting, LangGraph
Backend	APIs, bases de datos, autenticación, arquitectura	REST, FastAPI, SQL, JWT, microservicios
Data Science	Python, estadística, análisis, ML, visualización	pandas, scikit-learn, regresión, clustering
Frontend	HTML/CSS, JavaScript, frameworks, UX web	DOM, React, formularios, consumo de APIs
Cloud / DevOps	Cloud, Linux, contenedores, CI/CD, redes	OCI, Docker, Kubernetes, VCN, pipelines

Audiencias

Nivel	Perfil	Objetivo de adaptación
1	Escolar	Conceptos simples, analogías y mínima jerga.
2	Bachillerato / principiante tecnológico	Fundamentos y ejemplos concretos.
3	Universitario / transición de carrera	Conceptos técnicos con contexto y práctica.
4	Junior / Semi Senior	Detalle técnico, decisiones y buenas prácticas.
5	Senior / Arquitecto / especialista	Profundidad, trade-offs, arquitectura y precisión.
Transversal	Ejecutivo / no técnico	Impacto, riesgos, beneficios y decisiones sin exceso de implementación.

Principio clave

Un mismo documento técnico puede servir para varios perfiles. El conocimiento fuente se mantiene; lo que cambia es la transformación pedagógica del LLM. Esto simplifica la recopilación y permite demostrar el valor central de NuevaMente.

3. Recopilación y gobierno de datos

Antes de descargar documentos de forma masiva, crear un catálogo de fuentes. Para el MVP, una meta razonable es comenzar con 3-5 documentos por categoría y crecer hacia 10-20 documentos de alta calidad por categoría.

Estructura de carpetas sugerida

```
data/  
  raw/  
    ai/  
    backend/  
    data_science/  
    frontend/  
    cloud_devops/  
  processed/  
  metadata/document_catalog.csv  
  evaluation/rag_evaluation.csv  
  samples/
```

Campos mínimos de document_catalog.csv

Campo	Uso
document_id	Identificador único.
categoria	Área principal.
subcategoria	Subárea temática.
tema	Tema concreto del documento.
tecnologia	Tecnología o herramienta principal.
nivel_tecnico	básico / intermedio / avanzado.
idioma	Idioma del contenido.
formato	PDF / Markdown / TXT.
fuentes	Origen del documento.
licencia	Condiciones de uso o redistribución.
fecha_fuente	Vigencia del contenido.
estado	pendiente / aprobado / rechazado / procesado.

3.1 Criterios para aceptar una fuente

- Fuente identificable y preferentemente oficial o técnicamente confiable.
- Contenido suficientemente actualizado para su temática.
- Texto extraíble o procesable con calidad.
- Sin duplicados evidentes.
- Licencia o términos de uso registrados.
- Cobertura útil para preguntas reales del usuario.
- Nivel técnico identificable.

La meta inicial no es “tener muchos documentos”, sino disponer de un corpus trazable, vigente y útil para evaluar recuperación y generación.

4. Primer sprint: cómo empezar desde hoy

El objetivo de los primeros días es obtener un corpus pequeño que ya pueda atravesar la pipeline completa. No esperar a tener todos los documentos para comenzar a programar.

Momento	Acciones	Entregable
Día 1	Cerrar 5 categorías; definir subcategorías, niveles y esquema de metadatos.	Taxonomía v1 + plantilla de catálogo.
Día 2	Seleccionar 3-5 fuentes/documentos por categoría; registrar licencia y vigencia.	15-25 documentos aprobados.
Día 3	Implementar extracción PDF/MD/TXT y normalización básica.	Corpus raw + processed reproducible.
Día 4	Crear chunking v1 y metadatos por chunk.	Chunks trazables al documento origen.
Día 5	Generar embeddings, cargar Vector Store y ejecutar primeras búsquedas.	Primer retrieval funcional.
Día 6-7	Crear 20-30 preguntas manuales y medir recuperación.	Baseline de evaluación RAG.

Definition of Done del primer sprint

- Un documento puede cargarse y extraerse sin intervención manual.
- Cada chunk conserva document_id, categoría, tema y fuente.
- Una pregunta devuelve chunks relevantes del Vector Store.
- Existe un archivo de evaluación con al menos 20 preguntas.
- Los resultados de retrieval pueden reproducirse desde un notebook o script.

5. Evaluación del RAG

La evaluación debe crearse antes de optimizar. Un conjunto pequeño de preguntas conocidas permite comparar chunking, embeddings y parámetros de retrieval sin depender solo de impresiones subjetivas.

Estructura sugerida de rag_evaluation.csv

Campo	Ejemplo
question	¿Qué es una API REST?
expected_document_id	BE_REST_001
expected_topic	APIs / REST
category	Backend
difficulty	básica
reference_answer	Respuesta breve validada por el equipo.

Métricas recomendadas

Métrica	Qué mide
Recall@K	Si el documento o chunk relevante aparece entre los K recuperados.
Precision@K	Qué proporción de los K recuperados es realmente útil.
Context relevance	Qué tan pertinente es el contexto recuperado.
Faithfulness	Si la respuesta está sustentada por el contexto recuperado.
Pedagogical fit	Si lenguaje, profundidad y formato corresponden a la audiencia.

Experimentos mínimos

- Comparar 2-3 configuraciones de chunking, por ejemplo 500/50, 800/100 y 1200/150 tokens/overlap.
- Comparar como máximo 2 modelos de embeddings al inicio.
- Evaluar varios valores de Top-K, por ejemplo 3, 5 y 8.
- Registrar resultados en una tabla reproducible y elegir configuración por evidencia.

6. Plan de acción de 5 semanas

Semana	Objetivo	Entregables del equipo de Ciencia de Datos
1	Fundación	Taxonomía, catálogo, criterios de fuentes, corpus inicial, extracción y evaluación base.
2	RAG v1	Chunking, embeddings, Vector Store, retrieval y primeras métricas.
3	Integración	Contrato JSON, conexión con backend, adaptación por audiencia y formatos.
4	Optimización	Experimentos, fidelidad, pedagogía, casos límite y regresión.
5	Entrega	Congelamiento, pruebas finales, documentación, evidencia de métricas y demos.

Entregables técnicos sugeridos

Artefacto	Propósito
01_data_collection.ipynb	Explorar, validar y registrar fuentes/documentos.
02_preprocessing.ipynb	Extracción, limpieza, metadatos y chunking.
03_embeddings_retrieval.ipynb	Embeddings, Vector Store y consultas.
04_rag_evaluation.ipynb	Métricas y comparación de configuraciones.
src/ingestion/	Código reusable de ingestión y parsing.
src/retrieval/	Chunking, embeddings, indexado y búsqueda.
src/evaluation/	Métricas, tests y reportes.
data/metadata/document_catalog.csv	Inventario y gobierno de documentos.
data/evaluation/rag_evaluation.csv	Benchmark del equipo.
docs/data_science.md	Metodología, decisiones y resultados.

7. Organización del trabajo del equipo de Ciencia de Datos

El trabajo debe dividirse por frentes técnicos y mantenerse coordinado mediante un tablero común. Las responsabilidades pueden rotar según la carga de trabajo, pero cada frente debe tener tareas, criterios de aceptación y resultados verificables.

Frente	Responsabilidades
Data Collection	Fuentes, catálogo, licencias, vigencia y cobertura temática.
Preprocessing	Extracción, normalización, metadatos y chunking.
RAG / Retrieval	Embeddings, Vector Store, Top-K, filtros y trazabilidad.
Prompting / Adaptación	Perfiles de audiencia, formatos pedagógicos y estructura JSON.
Evaluation / QA	Benchmark, métricas, fidelidad, casos límite y regresión.
Integración	Contrato con backend, empaquetado y documentación técnica.

Ritmo de trabajo recomendado

- Daily de 10-15 minutos: terminado / siguiente / bloqueo.
- Demo técnica interna al menos una vez por semana.
- Pull Request obligatorio para código reusable.
- Cada experimento debe registrar configuración, fecha, corpus y resultado.
- No cambiar embeddings o chunking del sistema sin actualizar el benchmark.

Toda decisión técnica importante debe quedar documentada para que el equipo pueda reproducir el resultado y explicar por qué se eligió una configuración.

8. Checklist de Ciencia de Datos antes de la entrega

- | |
|---|
| ☐ Corpus con documentos de las 5 categorías seleccionadas. |
| ☐ Catálogo completo con fuente, licencia, nivel y estado. |
| ☐ Ingestión funcional de PDF, Markdown y TXT. |
| ☐ Chunks con metadatos y trazabilidad al documento original. |
| ☐ Embeddings y Vector Store reproducibles. |
| ☐ Retrieval evaluado con benchmark propio. |
| ☐ Respuesta RAG fundamentada en contexto recuperado. |
| ☐ Adaptación validada para múltiples audiencias. |
| ☐ Al menos 2 formatos pedagógicos probados. |
| ☐ Métrica o procedimiento de fidelidad documentado. |
| ☐ JSON de salida validado con backend. |
| ☐ Casos de prueba y regresión ejecutados. |
| ☐ README y documentación técnica actualizados. |
| ☐ Evidencia de experimentos lista para la presentación. |

Criterio de éxito

El sistema debe poder explicar por qué recuperó un contenido, de qué fuente provino y si la respuesta generada permanece fiel a esa fuente. La adaptación pedagógica debe cambiar el nivel y la forma de explicar, no inventar conocimiento nuevo.

Documento de trabajo interno - Equipo de Ciencia de Datos - NuevaMente